Báo cáo Đối chiếu: Tài liệu Kiến trúc Kỹ thuật v1.0 & Dự án Thực tế

Tài liệu gốc: **AI Compute Fabric — Kiến trúc kỹ thuật v1.0** (25/10/2026, gồm 16 mục, Phụ lục A–C và 2 sơ đồ kiến trúc) · Dự án: `ai-compute-fabric` (cập nhật & kiểm định toàn diện ngày 08/11/2026) · Ngôn ngữ: Tiếng Việt

Tóm tắt điều hành (Executive Summary).
1. **Mức độ hoàn thành tổng thể đạt 76.5%** (41 hạng mục Đã làm hoàn chỉnh, 2 hạng mục Làm theo phương án tối ưu hơn tài liệu, 18 hạng mục Một phần, và chỉ còn 7 hạng mục Chưa làm do thuộc phạm vi 3–4+ máy vật lý hoặc tính năng phụ).
2. **Toàn bộ lõi kỹ thuật 1–2 máy đã chạy thực tế và vượt chỉ tiêu nghiệm thu:** Cụm 2 GPU hỗn hợp (Linux RTX 3060 12 GB + Windows RTX 5070 12 GB) chạy mô hình `Qwen3-Coder-30B-A3B` đạt **154/164 bài HumanEval (93,9%)** với tốc độ **124,7 tok/s** (nhanh gấp **3,16×** và tiết kiệm **13,7% điện năng/token** so với chạy 1 card). Thời gian thu hồi GPU khi chủ máy dùng lại đạt **174–230 ms** (< 2.000 ms). Định tuyến KV Prefix Affinity giảm TTFT từ **1.390 ms xuống 38 ms (-97,3%)**.
3. **An ninh, Kiểm thử và Vận hành đã được khóa chặt:** Đã hoàn tất 2 vòng kiểm toán bảo mật OWASP ASVS L2 trên 14 bề mặt (0 lỗ hổng mở, 0 CVE, giết 8/8 mutant), tự động xoay vòng và nạp nóng chứng chỉ mTLS, loại bỏ hoàn toàn mật khẩu mặc định, nạp đường cong thực nghiệm vào Scheduler/Router, và vượt qua **181 Go tests + 825+ Python tests**.

41	2	18	7
Đã làm (60%)	Làm tối ưu hơn (3%)	Một phần (26%)	Chưa làm (10%)



Tổng số 68 hạng mục kỹ thuật trích xuất từ 16 mục của tài liệu. Điểm quy đổi = (Đã làm + Làm khác + 0,5 × Một phần) / Tổng số = **76.5%**. Các hạng mục được tính ngang trọng số; điểm phản ánh độ phủ yêu cầu, không phải mức sẵn sàng triển khai quy mô 200 PC.

1. Phương pháp đối chiếu và độ tin cậy của bằng chứng

- **[ĐO]**: Số liệu đo thực nghiệm trên phần cứng thật trong `benchmarks/reports/` hoặc kết quả chạy bộ kiểm thử tự động ngày 08/11/2026. **[ĐỌC MÃ]**: Đối chiếu trực tiếp từng tệp mã nguồn trong kho `ai-compute-fabric`. **[ƯỚC TÍNH]**: Nhận định tổng hợp phục vụ quản trị.
- **Kết quả chạy kiểm thử xác nhận ngày 08/11/2026:** Toàn bộ **5 module Go** (`core/vram-estimator`, `apps/worker-guard`, `apps/site-gateway`, `apps/global-control-plane`, `contracts/gen/go`) đạt **181/181 test pass** với `go test -race`; bộ kiểm thử Python (`unit`, `security`, `correctness`, `resilience`, `performance`, `audio_quality`) đạt **100% pass** (kèm 1 cổng `xfail(strict=True)` dành riêng cho chữ ký Pháp chế con người).
- **Quy mô mã nguồn thực tế (không tính test):** `apps/` ~12,2 nghìn dòng Go/JS, `agents/` ~5,6 nghìn dòng Python, `serverless/` ~3,1 nghìn dòng, `studio/` ~2,7 nghìn dòng, `bestofn/` ~1,8 nghìn dòng, `windows-agent/` ~1,5 nghìn dòng, `benchmarks/` ~16 nghìn dòng, `tests/` ~12,1 nghìn dòng.

2. Phân tích 2 sơ đồ kiến trúc nhúng trong tài liệu .docx

Sơ đồ trong .docx	Nội dung thiết kế trong tài liệu	Đối chiếu với thực tế triển khai
Hình 1 (Mục 2 — Kiến trúc tổng thể 5 tầng)	Sơ đồ gồm 5 tầng từ trên xuống: (1) GLOBAL CONTROL PLANE (Policy · Model Registry · Metering · Cross-Site Federation) → (2) ADAPTIVE COMPUTE SCHEDULER (Latency/Throughput/Cost Predictor · Lease Manager · Topology Builder) → (3) ENTERPRISE SITE GATEWAY (Worker Registry · Task Queue · KV/Weight Cache · Local Policy Engine) → (4) 4 hộp Runtime (QWEN INFERENCE, NATIVE MoE, AI AGENTS, AUDIO + TRAINING) → (5) PC WORKER FABRIC · 200 máy / LAN (Windows/Linux Agent · Hardware Profiler · Quota Guard · Local Cache) + nhánh phải WAN Federation.	Cả 5 tầng đều đã có thành phần chạy thật trong repo. Điểm cần lưu ý trong tài liệu gốc: chú thích dưới hình ghi 'Năm engine vận hành dưới một Fabric Scheduler' nhưng hình vẽ gộp Audio + Training vào hộp thứ 4 (tổng cộng 4 hộp runtime). Ngoài ra dự án thực tế đã mở rộng tầng thứ 4 thêm 3 khối lớn: Serverless GPU/Python, AI Studio và Best-of-N Code Orchestrator.

Sơ đồ trong .docx	Nội dung thiết kế trong tài liệu	Đối chiếu với thực tế triển khai
Hình 2 (Mục 3 — Luồng thực thi đầu-cuối: Online Serving vs Offline Training)	Sơ đồ tách bạch 2 nhánh từ ENGINE / WORKERS: nhánh trên (Online) đi từ AI USER REQUEST → EXECUTION PLAN → ENGINE / WORKERS → RESULT + TELEMETRY; nhánh dưới (Offline) đi từ ENGINE / WORKERS → TRAINING PIPELINE (LoRA · local steps, DDP / DiLoCo · registry) → RESULT + TELEMETRY.	Quy tắc cốt lõi của Hình 2 ('Training không nằm trong đường sinh token; chỉ phát hành model sau khi đánh giá chất lượng') được tuân thủ nghiêm ngặt trong studio/ và training/coordinator.py (hàm promote_if_passes kiểm tra cổng chất lượng trước khi xuất adapter GGUF sang llama-server). Phát hiện lỗi chế bản trong file .docx gốc: ảnh Hình 2 bị lỗi font chữ (hiện ký tự ô vuông □ tại 'LoRA □ local steps' và 'DDP / DiLoCo □ registry').

3. Đối chiếu chi tiết 68 hạng mục theo từng chương mục của tài liệu

Yêu cầu trong tài liệu	Trạng thái	Bằng chứng xác thực trong project
§2 Kiến trúc — 4 đã làm · 0 làm khác · 4 một phần · 0 chưa		
Site Gateway (Go + gRPC): registry, lease, link graph, policy, cache	**Đã làm**	apps/site-gateway: Register/Heartbeat, LeaseService, router WFQ + KV affinity + cost-aware ReferenceTokS + deadline, mTLS hot-reload, RBAC, audit chuỗi băm SHA-256 [ĐỌC MÃ]; 109 Go tests pass [ĐO]
PC Worker Agent: profiling, quota, idle, sandbox, signed job, heartbeat (Windows/Linux)	**Đã làm**	apps/worker-guard (NVML VRAM thật, thu hồi GPU <2s, heartbeat 6-bucket, Windows service, P-core pin) + agents/sl_security.py (HMAC-SHA256 signed job ServerlessSig, NonceCache chống replay) + agents/sandbox.py (bubblewrap/Docker --cpus, --memory, --pids-limit, AICF_VRAM_MB) [ĐỌC MÃ, ĐO]
Global Control Plane: chính sách, metering, federation	**Một phần**	apps/global-control-plane: mTLS 1.3, thu hồi serial, DataClass residency, sổ cái bất biến; 16 Go tests pass. Chưa chạy qua WAN/nhiều site vật lý thật [ĐỌC MÃ, ĐO]
Inference Runtime: llama.cpp là mặc định; Prima/MoLink đối chứng; KT/vLLM có điều kiện	**Một phần**	llama.cpp chạy production (split, smart_1gpu, merged RPC); vLLM/SGLang đã đo đối chứng kỹ (P10–P13); KTransformers đã đo micro-bench (P8). Prima/MoLink chưa tích hợp [ĐO]
Agent Runtime: LangGraph + Fabric task API	**Đã làm**	agents/orchestrator.py (LangGraph DAG + checkpoint), task_bus.py (NATS JetStream), dag_api.py [ĐỌC MÃ, ĐO]
Audio & Training: ASR/TTS, fine-tune, DiLoCo, đánh giá	**Một phần**	Có Qwen3-ASR-0.6B (WER 8,11%), LoRA/QLoRA, LoRA merger (4 thuật toán), mô phỏng DiLoCo vs DDP; TTS dùng mms-tts-vie (round-trip WER 1,82%), enhancement dùng Wiener spectral gate numpy [ĐO]
Observability & Data: Postgres, object storage, NATS, OpenTelemetry, Prometheus/Grafana	**Một phần**	Postgres/pgvector + NATS JetStream + /metrics Prometheus + deploy/monitoring/ (prometheus.yml, 6 cảnh báo SRE, alertmanager.yml, grafana_aicf_dashboard.json 4 panel). Chưa có OpenTelemetry tracing và S3/MinIO object storage [ĐỌC MÃ]
Ba mặt phẳng control / data / management (cert rotation, provisioning)	**Đã làm**	Control/data/management tách rõ; internal/pki hỗ trợ RotateLeaves + NeedsRotation (-rotate -renew-within-days 30) và mtls.go tự nạp nóng chứng chỉ mới (GetCertificate / GetClientCertificate) không cần khởi động lại [ĐỌC MÃ, ĐO]
§3 Luồng — 4 đã làm · 0 làm khác · 0 một phần · 0 chưa		
Luồng một yêu cầu Qwen: xác thực tenant, định tuyến, lease, stream, metrics, failover	**Đã làm**	/v1/chat/completions qua gateway → router → worker; test affinity, fairness, cost-aware routing, inference_load [ĐO]

Yêu cầu trong tài liệu	Trạng thái	Bằng chứng xác thực trong project
Định danh model bằng hash weights + tokenizer + revision	Đã làm	ModelIdentity (proto), core/model_registry/provenance.py, scheduler từ chối MODEL_IDENTITY_MISMATCH [ĐỌC MÃ, ĐO]
Workflow AI Agent → DAG task có kiểu → gộp kết quả	Đã làm	agents/ (15 typed task, validator), p5_agent_dag_e2e.json (DAG 14 bước) [ĐO]
Job fine-tune audio: manifest → kiểm quyền → tách speaker → train → cổng WER/CER → đăng ký artifact	Đã làm	audio/ingestion (manifest SHA-256 + consent_ref, speaker_disjoint_split 0% rò rỉ), training/coordinator.py (promote_if_passes cổng chất lượng + provenance registry), tests/audio_quality/ pass [ĐỌC MÃ, ĐO]
§4 Engine — 4 đã làm · 1 làm khác · 2 một phần · 2 chưa		
LOCAL_REPLICA: Qwen nhỏ trên từng PC, nhiều PC phục vụ song song	Đã làm	Chế độ split: Qwen3-4B 554,9 tok/s tổng với 4 luồng/máy [ĐO]; 2 node chạy song song scale 96,4% [ĐO]
LOCAL_HYBRID: GPU + RAM, expert nóng/lạnh cho MoE	Đã làm	smart_1gpu với -ncmoe; B06 trên RTX 3060 giảm 92% TPOT so với 32B dense offload; Qwen3-30B-A3B 49,9 tok/s (pp512/tg64) [ĐO]
DISTRIBUTED_MODEL bằng Prima.cpp / MoLink	Chưa làm	Chỉ có dòng trong upstream-manifest.yaml (license UNVERIFIED); không có adapter hay benchmark B04/B05 vì thiếu hỗ trợ Windows ổn định [ĐỌC MÃ]
DISTRIBUTED_MODEL: chạy được model vượt VRAM một máy bằng nhóm 2 node	Làm khác tài liệu	Làm bằng GGML RPC của llama.cpp (chế độ merged): Qwen3-30B-A3B ~135 tok/s, gấp 2,1× (ctx 8k) và 2,7× (ctx 32k) RTX 5070 đơn lẻ; Qwen3-Coder-30B-A3B đạt 154/164 HumanEval (93,9%) ở 124,7 tok/s [ĐO]. Tài liệu coi RPC chỉ là PoC
Nhóm 3–4 PC cùng một model	Chưa làm	Mới có 2 máy vật lý (RTX 3060 + RTX 5070); chưa có máy thứ 3–4 để đo thật
SPECULATIVE decoding	Một phần	B07: Qwen3-8B + draft 0,6B nhanh 1,185×, acceptance 0,49, nhưng đầu ra greedy giống baseline chỉ 25% [ĐO]. Giữ tắt ở production
Native MoE và distributed native MoE (bảo toàn phép tính)	Đã làm	Qwen3-30B-A3B và Coder-30B chạy native, merged 2 card; full-164 HumanEval 154/164 cho Coder-30B [ĐO]
Agent-level experts (router chọn Agent theo loại việc)	Đã làm	apps/site-gateway/compute_agents.go + router.go: nhãn năng lực (code, coder, reasoning, math, general), định tuyến theo X-AICF-Label / X-AICF-Agent + Best-of-N orchestrator [ĐỌC MÃ, ĐO]
KTransformers (cổng ISA, backend riêng)	Một phần	P8: đo baseline và micro-benchmark kt-kernel, cổng CPU ISA trong core/hardware_profiler; chưa tích hợp làm backend mặc định vì llama.cpp P-core + KV q8_0 đã đạt đích [ĐO]
§5 Scheduler — 4 đã làm · 0 làm khác · 2 một phần · 0 chưa		
Lọc cứng + chọn argmin hàm chi phí (bộ nhớ, identity, residency, SLA, năng lượng, rủi ro)	Đã làm	core/adaptive_scheduler/scheduler.py: lọc cứng + expected_score(w, r) = T_expected + lambda*resource_cost + mu*risk_penalty (§5.2), 18 unit tests pass [ĐỌC MÃ, ĐO]
Scheduler chi phí nói trên được cắm vào đường định tuyến thật	Đã làm	apps/site-gateway/router.go (pickPreferLocked chấm điểm (inflight+1)/ReferenceTokS kết hợp WFQ, KV affinity, priority, deadline); TestCostAwareRoutingPrefersFasterHardwareCurve pass [ĐỌC MÃ, ĐO]

Yêu cầu trong tài liệu	Trạng thái	Bằng chứng xác thực trong project
Đường cong benchmark theo (model, backend, phần cứng) + predictor học từ dữ liệu	Một phần	Đã nạp bảng MEASURED_CURVES (B01, B02, B06, full164) vào scheduler.py và ReferenceTokS vào compute_agents.go; chưa có mô hình ML tự học online từ log [ĐỌC MÃ, ĐO]
Lease có TTL, gia hạn bằng heartbeat; nhường PC cho người dùng	Đã làm	leases.go; thu hồi GPU trung vị 174 ms (Windows), 230 ms p50 / 257 ms p95 (Linux) < 2 s [ĐO]
Chống quá tải: quota tenant, giới hạn đồng thời, backpressure	Đã làm	Tenant fairness benchmark, MaxQueue, max_inflight, token bucket; test fair/leases pass [ĐO]
3 mức ưu tiên: người dùng > doanh nghiệp > federation	Một phần	Mức 1 (thu hồi GPU cho chủ máy) và mức 2 (interactive > batch, WFQ) đã đo thật; mức 3 (federation) có chính sách trong global-control-plane nhưng chưa đo tải thật liên site [ĐO]
§6 Agent — 7 đã làm · 0 làm khác · 1 một phần · 0 chưa		
Graph Orchestrator (LangGraph) + checkpoint	Đã làm	agents/orchestrator.py, task_bus.py [ĐỌC MÃ, ĐO]
Task Planner tách mục tiêu thành DAG kiểu	Đã làm	bestofn/planner.py (8 chiến lược planner) + agents/orchestrator.py dựng DAG 14 bước có kiểu [ĐỌC MÃ, ĐO]
Tool Registry (allowlist có kiểu) và không có shell tổng quát	Đã làm	agents/tool_registry.py; test an ninh pass [ĐO]
Task Executor + sandbox	Đã làm	agents/sandbox.py (bubblewrap/Docker); p7_pha0_sandbox_mutation.json giết 100% mutant [ĐO]
Knowledge Service: chunking, pgvector, rerank, cô lập tenant	Đã làm	agents/knowledge.py (RLS tenant_id, HNSW pgvector, rerank bge-reranker-v2-m3); p5_rag_quality_small_vi.json [ĐO]
Result Validator	Đã làm	agents/validator.py [ĐỌC MÃ, ĐO]
15 loại task chuẩn	Đã làm	tests/unit/test_15_typed_task_schemas.py [ĐO]
Ví dụ phân tích 10.000 cuộc gọi chạy thật	Một phần	Đã đo nạp streaming 10.000 file WAV (test_ten_thousand_files_stream_with_flat_memory, RSS tăng < 100 MB) và DAG E2E; chưa chạy 10.000 cuộc gọi qua ASR+LLM thật vì tốn ~30 giờ GPU [ĐO]
§7 Audio — 1 đã làm · 0 làm khác · 2 một phần · 1 chưa		
ASR bằng Qwen3-ASR	Đã làm	Qwen3-ASR-0.6B: WER sạch 8,11% (corpus 7,90%), CER 4,70% trên 100 clip FLEURS tiếng Việt [ĐO]
TTS bằng Qwen3-TTS	Một phần	Vòng TTS→ASR dùng facebook/mms-tts-vie (WER vòng lặp 1,82%, gắn cờ CC-BY-NC-4.0 phi thương mại), chưa dùng Qwen3-TTS [ĐO]
Enhancement bằng DeepFilterNet	Một phần	Đang dùng cổng phổ Wiener/spectral-gate numpy; giảm WER nhiễu 10 dB SNR từ 13,57% xuống 12,93% [ĐO]
Nhận diện sự kiện âm thanh (BEATs)	Chưa làm	Không có trong phạm vi hiện tại
§7 Training — 2 đã làm · 0 làm khác · 1 một phần · 1 chưa		
LoRA/PEFT một GPU	Đã làm	training/lora_trainer.py, QLoRA trong Studio, memory estimator sai số ≤ 9,5% so với thực đo [ĐO]
Quy tắc gộp LoRA không lấy trung bình A và B tùy tiện	Đã làm	training/lora_merger.py: hỗ trợ exact ΔW, svd, lora_fa; cảnh báo rõ độ lệch của naive [ĐỌC MÃ, ĐO]
DDP 2–4 GPU thật	Chưa làm	Chỉ có đối chứng 4 worker tuần tự trên một RTX 3060 (p6_diloco_vs_ddp.json)

Yêu cầu trong tài liệu	Trạng thái	Bằng chứng xác thực trong project
DiLoCo thử nghiệm	Một phần	p6_diloco_vs_ddp.json: giảm 95–97,5% băng thông WAN (H=20..40) với val loss 1,105 vs DDP 1,098 (+0,6%), nhưng mới mô phỏng tuần tự trên 1 GPU [ĐO]
§8 Hạ tầng — 2 đã làm · 0 làm khác · 2 một phần · 1 chưa		
PostgreSQL + pgvector	Đã làm	agents/knowledge.py, studio/ (psycopg, pgvector HNSW, RLS) [ĐO]
NATS JetStream (at-least-once, idempotent)	Đã làm	agents/task_bus.py, tests/integration/test_task_bus.py [ĐO]
Object storage cho checkpoint / model shard	Chưa làm	Dùng thư mục cục bộ có kiểm tra SHA-256; chưa tích hợp S3/MinIO
Model Registry có artifact ký số	Một phần	Hash SHA-256 weights+tokenizer+revision, provenance, chặn trust_remote_code và pickle RCE đã có; cosign mới ký SBOM/release manifest, chưa ký từng file GGUF [ĐỌC MÃ, ĐO]
Metrics/traces: OpenTelemetry, Prometheus, Grafana	Một phần	Có /metrics Prometheus, /api/stats, deploy/monitoring/ (Grafana dashboard 4 panel + 6 Prometheus alert rules + Alertmanager). Chưa có OpenTelemetry tracing [ĐỌC MÃ]
§9 WAN — 1 đã làm · 0 làm khác · 1 một phần · 1 chưa		
AdvertiseCapacity / OfferJob / AcceptJob / ReportUsage	Một phần	apps/global-control-plane/plane.go + 16 unit tests pass; triển khai qua HTTP/JSON + mTLS, chưa tách file proto riêng [ĐỌC MÃ, ĐO]
mTLS liên site, thu hồi chứng chỉ, DataClass không rời site gốc	Đã làm	plane.go, policy.go; gcp_test, pki_test pass [ĐO]
Pilot federation 2–3 site thật	Chưa làm	Chưa có 2–3 site vật lý; chỉ kiểm thử mức đơn vị và 1 site LAN (2 máy)
§10 Monorepo — 1 đã làm · 0 làm khác · 2 một phần · 1 chưa		
upstream-manifest: commit ghim, SPDX, người phụ trách	Một phần	third_party/upstream-manifest.yaml có đầy đủ commit/SPDX/owner; legal_reviewed=false đang chờ bộ phận Pháp chế ký duyệt (cổng xfail strict trong test_manifest_schema.py) [ĐỌC MÃ, ĐO]
NOTICE và SBOM ký cosign	Đã làm	NOTICE (liệt kê rõ Apache-2.0, MIT, CC-BY-4.0, CC-BY-NC-4.0), Makefile verify-release-v1, tests/security/test_sbom_signatures.py [ĐO]
third_party/patches và pinned-forks	Chưa làm	Thư mục third_party chỉ dùng upstream-manifest.yaml, không giữ bản fork nội bộ
Cấu trúc thư mục theo Phụ lục A	Một phần	Có apps/core/inference/agents/audio/training/contracts/benchmarks/serverless/studio/bestofn; một số thư mục khung (federation/, infrastructure/) để rỗng vì mã đặt trong apps/ và deploy/ [ĐỌC MÃ]
§11 Contract — 3 đã làm · 1 làm khác · 0 một phần · 0 chưa		
Protobuf: WorkerCapacity, ModelIdentity, WorkloadJob, Lease; Fleet + Lease service	Đã làm	contracts/protobuf/aicf/v1 + golden test pass [ĐO]
Service Placement, Inference, AgentJobs, Training, Federation (proto)	Làm khác tài liệu	Hiện thực đầy đủ qua OpenAPI v1 REST + Go/Python typed schema thay vì nhân bản thêm 5 file gRPC proto không cần thiết cho cụm 2 máy [ĐỌC MÃ]
OpenAPI v1 kiểm tự động theo route	Đã làm	contracts/openapi/aicf_v1.yaml + test_openapi_routes.py [ĐO]
Engine lifecycle 7 hàm (probe…stop) + adapter	Đã làm	inference/engine.py; adapter llama_cpp, vllm (kèm hợp đồng kiểm thử p3_vllm_adapter_contract.json) [ĐO]
§13 An ninh — 3 đã làm · 0 làm khác · 1 một phần · 0 chưa		

Yêu cầu trong tài liệu	Trạng thái	Bằng chứng xác thực trong project
mTLS, RBAC, audit, tenant isolation	Đã làm	mtls.go (tự nạp nóng cert), rbac.go, audit.go; rà soát OWASP ASVS L2 qua 14 bề mặt (S1–S14) với 0 lỗ hổng mở [ĐO]
Cấm generic shell, allowlist tool, sandbox, không trust_remote_code	Đã làm	tool_registry, sandbox, model_registry/security.py; mutation test giết 8/8 mutant (100%) [ĐO]
Kiểm toán giấy phép model Qwen, dataset giọng nói, mã nguồn	Một phần	Đã kiểm toán kỹ thuật trong NOTICE, studio/audio_service.py (cảnh báo CC-BY-NC-4.0 cho mms-tts-vie) và test_manifest_schema.py; chờ chữ ký Pháp chế con người (legal_reviewed=false) [ĐỌC MÃ, ĐO]
Không để LAN mặc định tin cậy / không lộ RPC	Đã làm	run_demo.sh tự sinh mật khẩu & token ngẫu nhiên (0600) và tự xoay vòng chứng chỉ mTLS trước hạn 30 ngày; apps/worker-guard/rpc_proxy.go bọc GGML RPC bằng IP allowlist + giới hạn kết nối [ĐỌC MÃ, ĐO]
§15 Kiểm thử — 3 đã làm · 0 làm khác · 0 một phần · 0 chưa		
Bộ kiểm thử theo tầng (unit, integration, correctness, resilience, security)	Đã làm	181 Go tests pass trên 5 module; 825+ pytest cases trên toàn repo (unit, security, correctness, resilience, performance, audio_quality pass 100%) [ĐO 08/11]
Performance và audio-quality trong thư mục tests/	Đã làm	tests/performance/test_performance_gates.py (7 cổng B01/B06/B07/G-RECLAIM/DiLoCo/Prefix/Scheduler) và tests/audio_quality/test_audio_quality_gates.py (5 cổng WER/CER/speaker-split/SNR) pass 100% [ĐO 08/11]
CI tự động	Đã làm	.github/workflows/ci.yml, Makefile verify-release-v1 [ĐỌC MÃ, ĐO]
§14 Deliverable — 2 đã làm · 0 làm khác · 0 một phần · 0 chưa		
ADR cho từng quyết định lớn	Đã làm	docs/adr: 4 ADR (0001 GPUStack, 0002 Event Bus, 0003 Serverless GPU, 0004 Web Editor) [ĐỌC MÃ]
Machine inventory + baseline vLLM TP/PP	Đã làm	p0_probe_*.json; P10–P13 (vLLM/SGLang PP, 2 GPU) [ĐO]

4. Ma trận Benchmark B01–B09 (§12.4) và Cổng nghiệm thu hiệu năng (§12.5)

Mã	Tài liệu yêu cầu	Trạng thái	Kết quả thực tế trong dự án
B01	1 PC, Qwen nhỏ: TTFT, TPOT, joule/token, VRAM	Đã làm	llama-bench Qwen3-4B (69,5–122,9 tok/s) & Qwen3-8B (43,8–82,2 tok/s) trên RTX 3060 và RTX 5070; joule/token đo bằng NVML (sai số −1,9% / +0,1%)
B02	1 PC, Qwen lớn + offload RAM	Đã làm	Qwen2.5-32B Q4_K_M offload trên RTX 3060: 3,97 tok/s, TPOT 252 ms (p0_B02) — làm mốc chứng minh nghẽn PCIe
B03	3–4 PC, vLLM TP/PP baseline	Một phần	Đo kỹ trên 2 GPU (P10–P13): vLLM/SGLang PP qua WSL2 bị nghẽn NAT/NCCL; chứng minh llama.cpp GGML RPC vượt trội trên cụm hỗn hợp Linux+Windows
B04	3–4 PC, Prima.cpp	Chưa làm	Không triển khai vì thiếu hỗ trợ Windows ổn định và GGML RPC đã đạt 135 tok/s
B05	3–4 PC, MoLink	Chưa làm	Không triển khai vì trùng mục tiêu với GGML RPC trên cụm 2 node
B06	Qwen MoE local / hybrid	Đã làm	p4_B06: Qwen3-30B-A3B với n-cpu-moe 48→18 trên 1× RTX 3060 đạt 49,87 tok/s (TPOT 20,1 ms, giảm 92,0% so với B02)
B07	Speculative target/draft	Đã làm	p4_B07: Qwen3-8B + draft Qwen3-0.6B đạt 75,29 tok/s (1,185×), acceptance 0,49; phát hiện độ lệch greedy 25% nên giữ tắt ở production
B08	8–40 PC, Agent task độc lập	Một phần	Thật trên 2 máy đạt hiệu suất ghép 96,4%; mô phỏng 8 / 20 / 40 worker đạt 85,5% / 74,1% / 56,8% (ghi rõ nhãn SIMULATED)
B09	4–8 PC, audio training DDP/DiLoCo	Một phần	p6_diloco_vs_ddp.json: đo 4 worker tuần tự trên 1 GPU, chứng minh DiLoCo giảm 95–97,5% băng thông WAN với độ lệch loss +0,6%

Đối chiếu các Cổng nghiệm thu hiệu năng (Performance Gates — §12.5)

Chỉ tiêu nghiệm thu trong tài liệu	Kết quả thực đo & Kiểm chứng tự động
Distributed Qwen: giảm TPOT ≥30% so với PP baseline, hoặc chạy được model không vừa máy đơn	ĐẠT VƯỢT MỨC: Qwen3-30B-A3B (18,6 GB, vượt VRAM 12 GB của 1 card) chạy 100% trong VRAM 2 card (RTX 3060 + RTX 5070) đạt ~135 tok/s (8k) và 124,7 tok/s ở full-164 HumanEval (154/164 = 93,9%, nhanh gấp 3,16× chế độ 1 card 39,5 tok/s) [ĐO]
MoE Hybrid (G-MOE-HYBRID): giảm TPOT ≥40% so với dense offload tương đương	ĐẠT VƯỢT MỨC: B06 giảm 92,0% TPOT (20,1 ms vs 252,0 ms của B02) [ĐO, kiểm tự động trong tests/performance/]
KV Prefix Affinity (G-PREFIX): giảm TTFT ≥30% cho hội thoại nhiều lượt	ĐẠT VƯỢT MỨC: TTFT giảm từ 1.390 ms xuống 38 ms (giảm 97,3%) cho ngữ cảnh ~2.000 token nhờ định tuyến ưu tiên worker giữ KV cache [ĐO]
Task parallel: tổng throughput ≥80% tổng các node độc lập	ĐẠT Ở 2 NODE THẬT (96,4%, 554,9 tok/s tổng); ở 40 worker mô phỏng đạt 56,8% do nghẽn điều phối trung tâm [ĐO]
Ưu tiên tài nguyên (G-RECLAIM): không chiếm ngoài quota, thu hồi GPU < 2,0 giây	ĐẠT VƯỢT MỨC: Thu hồi GPU trung vị 174 ms trên Windows (10/10 lần) và p50 = 230 ms / p95 = 257,1 ms trên Linux (< 2.000 ms) [ĐO]
Audio & Training (G-AUDIO / G-DILOCO): WER tiếng Việt ≤10%, DiLoCo giảm ≥95% băng thông WAN	ĐẠT: Qwen3-ASR-0.6B đạt WER sạch 8,11% (corpus 7,90%) trên FLEURS vi_vn; TTS→ASR round-trip WER 1,82%; DiLoCo H=20..40 giảm 95–97,5% lưu lượng WAN [ĐO]

5. Hiệu quả Kinh tế & Năng lượng thực đo (TCO / Joule trên mỗi Token)

Đo trực tiếp bằng cảm biến phần cứng NVML trên cụm RTX 3060 + RTX 5070 qua bộ 164 bài toán lập trình HumanEval (quy đổi theo đơn giá điện kinh doanh 3.000 VNĐ/kWh). Chỉ tính công suất board GPU, chưa gồm CPU/RAM/điện toàn hệ thống; cấu hình 1 card không cộng điện chờ của card còn lại (2–12 W). Nguồn: benchmarks/reports/full164/*.json:

Cấu hình vận hành	Tốc độ sinh	Tiêu thụ điện	Độ chính xác	Chi phí điện / 1M token	Nhận xét vận hành
Qwen3-Coder-30B-A3B (Ghép 2 GPU: RTX 3060 + RTX 5070, merged RPC)	124,7 tok/s	1,61 J/tok	154/164 (93,9%)	~1.340 VNĐ / 1M token	Cấu hình chủ lực: chạy 100% trên VRAM 2 card, vừa nhanh gấp 3,16× vừa tiết kiệm 13,7% điện/token so với chạy 1 card.
Qwen3-Coder-30B-A3B (1 GPU: RTX 3060 smart_1gpu - ncmoe)	39,5 tok/s	1,86 J/tok	154/164 (93,9%)	~1.550 VNĐ / 1M token	Giữ nguyên chất lượng 93,9% khi chỉ còn 1 máy rảnh, tốc độ chỉ bằng ~32% nhưng điện/token chỉ cao hơn ~16% (CPU gánh các tầng MoE lạnh).
Qwen3-8B Q4_K_M (full-164, chế độ split)	63,0 tok/s	2,79 J/tok	134/164 (81,7%)	~2.325 VNĐ / 1M token	Phù hợp chat tương tác và bước nháp; ghép 2 card chỉ nhanh hơn ~15% (72,2 tok/s, 3,01 J/tok).
Qwen3-4B Q4_K_M (full-164, chế độ split)	100,2 tok/s	1,74 J/tok	128/164 (78,0%)	~1.450 VNĐ / 1M token	Chi phí thấp nhất, phù hợp LOCAL_REPLICA.

6. Tiến độ Lộ trình P0–P6 so với kế hoạch

Giai đoạn	Trạng thái	Kết quả đạt được & Ghi chú
P0 Audit & baseline	**Đã làm**	Probe phần cứng, baseline llama.cpp và vLLM/SGLang, kiểm toán kỹ thuật giấy phép (NOTICE + upstream-manifest) đều hoàn tất.
P1 Worker Fabric	**Một phần**	Agent, Gateway, registry, lease, NATS, adapter llama.cpp, mTLS xoay vòng tự động đều hoàn tất. Cổng '5–20–40 node vật lý' mới kiểm trên 2 máy thật + 40 worker mô phỏng.
P2 Distributed Qwen	**Đã làm**	Topology coordinator + chế độ merged qua GGML RPC (kèm rpc_proxy lọc IP) chạy ổn định ở 135 tok/s (8k) và 124,7 tok/s (full-164), vượt xa mục tiêu.

Giai đoạn	Trạng thái	Kết quả đạt được & Ghi chú
P3 MoE & AI Agent	Đã làm	MoE native/hybrid, LangGraph DAG, RAG pgvector RLS, ASR, tool allowlist, sandbox, định tuyến theo nhãn năng lực (X-AICF-Label) đều đạt.
P4 Audio & DiLoCo	Một phần	ASR Qwen3 (WER 8,11%), LoRA/QLoRA, LoRA merger, cổng kiểm thử tests/audio_quality/ hoàn tất. DiLoCo mới dừng ở mô phỏng 1 GPU.
P5 Pilot 200 PC / WAN	Một phần	Đã hoàn thành rà soát bảo mật nội bộ 2 vòng (OWASP ASVS L2, 14 bề mặt, 0 lỗ hổng mở, 8/8 mutant bị tiêu diệt). Chưa có dàn 200 PC và 2–3 site WAN vật lý.
P6 Production hardening	Đã làm	Release manifest v1.0/v1.1, SBOM ký cosign, runbook SRE, Prometheus/Grafana monitoring, tự xoay vòng mTLS, loại bỏ mật khẩu mặc định, CI tự động.

7. Giá trị gia tăng dự án đã xây dựng vượt ngoài tài liệu v1.0

Hạng mục mở rộng	Quy mô & Bằng chứng nghiệm thu
Serverless GPU kiểu Modal	serverless/ + agents/sl_*.py (~3,1 nghìn dòng Python), SDK Python (~1,5 nghìn dòng), ký HMAC-SHA256 + chống replay, 14 file test (184 ca), ADR-003
AI Studio: RAG, QLoRA, ước tính full-FT, prompt/few-shot, Audio UI	studio/ (~2,7 nghìn dòng), RLS multi-tenant, tests/studio 45 ca
Best-of-N cho viết code + 8 chiến lược Planner + API cho IDE	bestofn/ (~1,8 nghìn dòng), P7_BEST_OF_N.md, tăng điểm HumanEval lên 93,9%+
Web IDE (CodeMirror) và Dashboard điều hành 200-GPU	apps/site-gateway/web (~8,8 nghìn dòng JS/HTML), topology switch trực quan, ADR-004
Giả lập Ollama + OpenAI Responses, trang kích hoạt 1 chạm, cầu nối đa nền tảng	ollama_emulator.go (~1,3 nghìn dòng), bridge/*.tmpl, connect.html; tương thích sẵn với Codex CLI, GJC, Cursor, Continue
Windows Agent (điều khiển ứng dụng desktop qua UIA)	windows-agent/ (~1,5 nghìn dòng), đối sánh thực nghiệm với UFO2, goose, UI-TARS
Benchmark HumanEval 164 bài × 23 cấu hình + đo điện NVML + kiểm định McNemar-Holm	benchmarks/reports/full164, 3 báo cáo chuyên sâu + PDF tổng hợp BAO_CAO_TONG_HOP_AI_VIET_CODE.pdf
Tối ưu CPU P-core affinity / KV cache Q8_0 / ncmoe & chẩn đoán sâu SGLang PP	P9, P12, P13 và hệ thống báo cáo kỹ thuật

8. Đánh giá tổng thể: Điểm mạnh & Điểm lệch giữa Tài liệu và Thực tế

Điểm mạnh của tài liệu đã phát huy hiệu quả

- Giữ kỷ luật bằng chứng nghiêm ngặt: tách rõ thiết kế với kết quả thực nghiệm, gắn nhãn [ĐO] / [ĐỌC MÃ] / [ƯỚC TÍNH], và đòi hỏi cùng model/tokenizer/quantization khi so sánh.
- Bốn nguyên tắc kiến trúc cốt lõi hoàn toàn chính xác khi va chạm thực tế: (1) chỉ phân tán để chạy vừa model lớn hoặc giảm TPOT; (2) ưu tiên MoE hybrid trước khi gọi mạng; (3) không truyền activation theo từng token qua WAN; (4) tuyệt đối không cộng trung bình ma trận A và B của LoRA.
- Dự báo đúng nút thắt phần cứng (RTX 3060 12 GB, băng thông LAN và PCIe) và thiết kế cơ chế nhường máy cho người dùng (Worker Guard thu hồi VRAM < 2 giây) rất sát nhu cầu doanh nghiệp.
- Thiết kế sẵn ma trận B01–B09 và các cổng nghiệm thu định lượng (§12.4–12.5), tạo khung chuẩn cho bộ test tự động trong tests/performance/ và tests/audio_quality/.
- Kiến trúc bảo mật phòng thủ theo chiều sâu (§13) giúp dự án vượt qua 2 vòng rà soát OWASP ASVS L2 trên 14 bề mặt với 0 lỗ hổng mở.

Những điểm thực tế triển khai khác với giả định ban đầu

Chủ đề	Phân tích nguyên nhân & Bài học thực tế
Chưa lường hết rào cản mạng trên Windows/WSL2	Tài liệu giả định các worker liên kết dễ dàng như cụm Linux thuần. Thực tế vLLM/SGLang đa nút bị chặn bởi NAT của WSL2 trên Windows 10 và lỗi khởi tạo NCCL (P10–P13), còn Prima/MoLink thiếu bản dựng Windows ổn định.
Đánh giá thấp GGML RPC của llama.cpp	Tài liệu xếp GGML RPC ở mức 'proof-of-concept'. Thực nghiệm chứng minh khi kết hợp với rpc_proxy lọc IP và chia tỷ lệ tensor hợp lý (52%/48%), GGML RPC là phương án phân tán duy nhất chạy mượt cả Linux + Windows, đạt 135 tok/s và 93,9% HumanEval.

Chủ đề	Phân tích nguyên nhân & Bài học thực tế
Danh sách engine ứng viên quá dàn trải	Nêu hơn 7 engine (llama.cpp, Prima, MoLink, vLLM, SGLang, KTransformers, Exo, Parallax). Thực tế chỉ cần llama.cpp làm trụ cột sản xuất và vLLM/SGLang làm mốc đối chứng.
Giả định sẵn quy mô 200 PC ngay từ đầu	Nhiều chỉ số nghiệm thu yêu cầu 40–200 PC vật lý và 2–3 site WAN. Tài liệu v2 nên tách rõ 'Cấp độ 1: Cụm 2–8 PC phòng ban (Production-ready)' và 'Cấp độ 2: Liên kết 40–200 PC toàn công ty'.
Chưa bao quát lớp sản phẩm ứng dụng	Một phần đáng kể giá trị thực tế của dự án nằm ở Serverless GPU (kiểu Modal), AI Studio, Best-of-N Code, Web IDE và trình giả lập Ollama cho công cụ lập trình — những phần này phát sinh trong quá trình làm thực tế.

9. Quản trị rủi ro hiện tại và đề xuất bước tiếp theo

Mức độ	Hạng mục rủi ro	Chi tiết & Trạng thái kiểm soát
Trung bình	**Chờ ký duyệt pháp lý con người (Legal Sign-off)**	Rà soát kỹ thuật về giấy phép (Apache-2.0 cho Qwen3, MIT cho BGE, cảnh báo CC-BY-NC-4.0 cho mms-tts-vie) đã hoàn tất trong NOTICE và studio/audio_service.py; trường legal_reviewed=false trong upstream-manifest.yaml đang chờ bộ phận Pháp chế ký xác nhận trước khi thương mại hóa.
Trung bình	**Chưa kiểm chứng vật lý ở quy mô ≥ 3 node và đa site WAN**	Mọi phép đo phần cứng thực tế hiện chạy trên 2 node (Linux RTX 3060 + Windows RTX 5070); kết quả 8–40 node và DiLoCo đa worker đang dựa trên mô phỏng.
Trung bình	**VRAM sát trần khi chạy model 30B ngữ cảnh dài (20k–32k)**	Ở chế độ ghép 2 card với ngữ cảnh 20.480 token, RTX 3060 dùng 11,2/12,3 GB và RTX 5070 dùng 11,5/12,2 GB [ĐO]. Cần giữ kỷ luật evict serverless container trước khi nạp model 30B.
Thấp	**Speculative decoding thay đổi phân phối đầu ra ở chế độ greedy**	B07 đạt tăng tốc 1,185× nhưng chỉ khớp 25–33% chuỗi greedy so với baseline; đã giữ ở trạng thái tắt mặc định.
Đã xử lý	**Mật khẩu mặc định, xoay vòng mTLS, nạp đường cong Scheduler & cổng test**	Đã khắc phục triệt để ngày 08/11/2026: run_demo.sh tự sinh mật khẩu/token ngẫu nhiên (0600); pki hỗ trợ -rotate + nạp nóng chứng chỉ mTLS không gián đoạn; nạp MEASURED_CURVES + ReferenceTokS vào Scheduler/Router; bổ sung đầy đủ tests/performance/ và tests/audio_quality/.

Kiến nghị hành động (Action Items)

1. Trình bộ phận Pháp chế ký duyệt third_party/upstream-manifest.yaml (chuyển legal_reviewed sang true sau khi phê duyệt) và thay thế mms-tts-vie (CC-BY-NC-4.0) nếu đưa tính năng TTS vào gói thương mại.
2. Bổ sung 1–2 PC vật lý tiếp theo (nâng lên 3–4 node) để đo nghiệm thu thực tế nhóm 3–4 máy (B03/B08) và thử nghiệm liên kết 2 site qua mạng nội bộ.
3. Ban hành Tài liệu Kiến trúc Kỹ thuật v2.0 cập nhật theo thực tế triển khai: chuẩn hóa GGML RPC (kèm rpc_proxy) làm đường phân tán chính cho cụm 2–4 PC hỗn hợp Windows/Linux, và đưa Serverless GPU, AI Studio, Best-of-N, Ollama Emulator vào kiến trúc chính thức.
4. Tích hợp thêm OpenTelemetry exporter và kho lưu trữ S3/MinIO khi mở rộng từ cụm phòng ban lên cụm liên chi nhánh.